

3. **Валидация ввода:** ровно 4 цифры, все цифры разные, только цифры (без букв/символов).
При некорректном вводе — сообщение об ошибке, попытка не засчитывается.
4. После каждой попытки в списке (историю) добавляется строка вида: 1234 → 1 бык, 2 коровы.
5. Счётчик количества попыток.
6. Проверка на победу: если «быков» равно 4 — игра завершена, показывается сообщение «Победа! Угадано за N попыток» и поле ввода блокируется до нажатия «Новая игра».
7. Кнопка «Новая игра» — сбрасывает состояние (загаданное число, историю попыток, счётчик).

Технические требования (обязательные):

- Логика генерации числа, сравнения и подсчёта быков/коров — написать самостоятельно (без готовых прт-библиотек).
- Использовать функции: отдельная функция для генерации числа, отдельная — для валидации ввода, отдельная — для подсчёта быков и коров. Не писать всё одним большим обработчиком клика.
- Файлы: `task4/index.html`, `task4/style.css`, `task4/script.js` (Canvas в этой лабораторной не нужен, обычный DOM).
- История попыток должна рендериться из массива данных (как в лабораторной №3), а не добавляться напрямую в `innerHTML` без хранения состояния.

Что сдавать: папку `task4/`.

Критерии проверки: корректность подсчёта быков/коров на разных наборах входных данных (преподаватель проверит вручную несколькими комбинациями), валидация ввода работает, есть возможность сыграть заново без перезагрузки страницы.

Лабораторная работа №5. Игра Simon (Симон говорит)

Цель: освоить работу с таймерами/задержками, звуком (по желанию) и последовательностями событий.

Референс: можно ориентироваться на freesimon.org, но верстка/цвета — свои, не копировать 1-в-1 чужой код.

Правила игры: на экране 4 цветные кнопки/сектора. Компьютер «показывает» последовательность (по очереди подсвечивает кнопки с паузой между ними). Игрок должен повторить последовательность в том же порядке, нажимая на кнопки. Если повторил верно — добавляется ещё один шаг к последовательности, и раунд повторяется заново (с начала, с уже более длинной последовательностью). Если ошибся — игра заканчивается, показывается достигнутый результат (длина последовательности).

Что нужно сделать:

1. Разметка: 4 кликабельных сектора/кнопки разного цвета, расположенных как единая фигура (круг из 4 частей или сетка 2×2) — оформление на Flexbox/Grid, как удобно.
2. Кнопка «Старт».
3. По нажатию «Старт»: последовательность из одного случайного элемента (0–3) генерируется и «проигрывается» — по очереди кнопки подсвечиваются (например, класс `.active` на время ~500мс) с паузой между ними.
4. После показа игрок кликает по кнопкам, повторяя последовательность. Клики должны блокироваться/игнорироваться, пока идёт показ последовательности компьютером (чтобы нельзя было кликать во время анимации).
5. Если игрок ввёл верную последовательность целиком — генерируется следующий раунд: к последовательности добавляется ещё один случайный элемент, и она снова показывается целиком с начала (по классике игры Simon).
6. Если игрок ошибся на любом шаге — игра завершается, выводится счёт («Вы дошли до уровня N»), кнопки блокируются до повторного нажатия «Старт».
7. Отображение текущего уровня/раунда на экране.

Технические требования (обязательные):

- Для показа последовательности с задержками использовать `setTimeout` (лично реализовать очередь показа — например, рекурсивно или через `async/await` с `Promise` + `setTimeout` внутри). Использование `setInterval` без остановки — так, чтобы таймер продолжал тикать после ошибки/после уничтожения игры — не допускается: все таймеры должны корректно завершаться.
- Состояние игры (текущая последовательность, ввод игрока, флаг «идёт показ / ждём ввода») хранить в переменных/объекте, а не «угадывать» его через CSS-классы на странице.
- Файлы: `task5/index.html`, `task5/style.css`, `task5/script.js`.
- Звук — по желанию (плюс к оценке, не обязателен). Если добавляете — использовать короткие аудиофайлы или `AudioContext`, звук не должен блокировать логику игры.

Что сдавать: папку `task5/`.

Критерии проверки: последовательность корректно удлиняется на 1 каждый успешный раунд, во время показа клики не засчитываются, игра корректно завершается при ошибке без «зависших» таймеров (открыть консоль и проверить, что после нескольких игр подряд нет наложения старых и новых последовательностей).

Лабораторная работа №6. Игра на Canvas («Змейка»)

Цель: освоить рисование на Canvas, игровой цикл через `requestAnimationFrame` и обработку клавиатуры.

Что нужно сделать: Реализовать классическую «Змейку»: